

Manual Técnico - Sistema de Gestión de Citas de Bienestar SENA

Documentación para Desarrolladores y Administradores

30/06/2026

1. Introducción

Este manual documenta la arquitectura, diseño y componentes del Sistema de Gestión de Citas de Bienestar SENA. Está dirigido a desarrolladores, arquitectos de software y personal técnico.

1.1 Propósito

- Documentar la arquitectura técnica completa
- Describir el stack tecnológico y dependencias
- Explicar el diseño de la base de datos
- Detallar patrones de diseño empleados
- Proporcionar guías de instalación y despliegue
- Establecer convenciones de código

2. Stack Tecnológico

Capa	Tecnología	Versión
Framework UI	React	19.2.4
Build	Vite	8.0.4
Enrutamiento	React Router DOM	7.14.0

Capa	Tecnología	Versión
Backend/BBDD	Supabase	2.103.0
Formularios	React Hook Form + Zod	7.72.1 / 4.3.6
Gráficos	Recharts	3.8.1
Fechas	date-fns	4.1.0
Iconos	Lucide React	1.8.0
Notificaciones	Sonner	2.0.7
Testing	Vitest + Testing Library	4.1.8
Linting	ESLint 9	9.39.4

3. Arquitectura del Sistema

3.1 Capas

- Presentación: Componentes React con CSS vanilla
- Lógica de Negocio: Custom hooks y validaciones Zod
- Acceso a Datos: Repository pattern con Supabase
- Backend: PostgreSQL, Auth, RLS, Realtime

3.2 Estructura de Carpetas

- src/features/ - Módulos: auth, appointments, dashboard, admin
- src/features/[feature]/api/ - Repositorios de datos
- src/features/[feature]/components/ - Componentes del módulo
- src/features/[feature]/hooks/ - Custom hooks
- src/features/[feature]/pages/ - Páginas/rutas
- src/features/[feature]/validations/ - Esquemas Zod

- src/shared/components/ - Componentes reutilizables
- src/shared/styles/ - Estilos y variables CSS
- src/providers/ - Context providers (Auth, Theme)
- src/routes/ - Configuración de rutas
- src/lib/ - Configuración de Supabase

3.3 Patrones Arquitectónicos

- Repository Pattern: Queries encapsuladas en clases repository
- Custom Hooks: Hooks que gestionan estado y orquestan repositories
- Enrichment en Repository: Unión de datos en capa de acceso
- Lazy Loading: React.lazy() para code splitting
- Realtime: Suscripciones Supabase postgres_changes
- Optimistic UI: Actualizaciones inmediatas en estado local

4. Base de Datos

4.1 Tabla profiles

Columna	Tipo	Descripción
id	UUID	PK, FK !' auth.users
full_name	text	Nombre completo
document_number	text	Número de documento
email	text	Correo electrónico
role_id	integer	FK !' roles
dependency_id	integer	FK !' dependencies
is_active	boolean	Estado de activación

Columna	Tipo	Descripción
created_at	timestampz	Fecha de creación
updated_at	timestampz	Última actualización

4.2 Tabla roles

Columna	Tipo	Descripción
id	integer	PK, SERIAL
name	text	UNIQUE: APRENDIZ, PROFESIONAL.
description	text	Descripción del rol
permissions	JSONB	Permisos por categoría

4.3 Tabla dependencies

Columna	Tipo	Descripción
id	integer	PK, SERIAL
name	text	Nombre de la dependencia
color	text	Color hexadecimal

4.4 Tabla appointments

Columna	Tipo	Descripción
id	UUID	PK
user_id	UUID	FK !' profiles (aprendiz)
professional_id	UUID	FK !' profiles (profesional)
dependency_id	integer	FK !' dependencies

Columna	Tipo	Descripción
scheduled_date	date	Fecha programada
scheduled_time	time	Hora programada
status	text	pending, confirmed, completed, cancelled, no_show
reason	text	Motivo de la consulta
notes	text	Notas adicionales

4.5 Tabla professional_schedules

Columna	Tipo	Descripción
id	UUID	PK
professional_id	UUID	FK !' profiles
day_of_week	integer	0=domingo, 6=sábado
start_time	time	Hora de inicio
end_time	time	Hora de fin
is_active	boolean	Estado activo/inactivo

4.6 Tabla clinical_notes

Columna	Tipo	Descripción
id	UUID	PK
appointment_id	UUID	FK !' appointments (CASCADE)
professional_id	UUID	FK !' profiles (CASCADE)
content	text	Contenido de la nota
is_private	boolean	Privada por defecto

4.7 Tabla audit_logs

Columna	Tipo	Descripción
id	UUID	PK
user_id	UUID	FK !' profiles
action	text	Tipo de acción
entity_type	text	Entidad afectada
entity_id	text	ID de la entidad
old_data	JSONB	Datos antes del cambio
new_data	JSONB	Datos después del cambio
user_agent	text	Navegador del usuario

4.8 Tabla system_config

Columna	Tipo	Descripción
key	text	PK - Clave de configuración
value	text/JSON	Valor de configuración

4.9 Relaciones

- auth.users 1!'1 profiles
- profiles N!'1 roles
- profiles N!'1 dependencies
- appointments N!'1 profiles (user_id)
- appointments N!'1 profiles (professional_id)
- appointments N!'1 dependencies
- professional_schedules N!'1 profiles

- clinical_notes N!1 appointments (CASCADE)
- audit_logs N!1 profiles

4.10 Row Level Security

- Profesionales: Solo ven sus propios horarios y notas
- Administradores: Acceso total a todas las tablas
- Coordinadores: Ven notas de su dependencia
- Aprendices: Solo ven/crean sus propias citas

5. Autenticación y Autorización

5.1 Flujo de Autenticación

- AuthProvider gestiona user, profile, loading, error
- Verifica sesión con getSession() al montar
- Suscribe a onAuthStateChanged() para eventos SIGNED_IN, PASSWORD_RECOVERY, SIGNED_OUT
- fetchProfile() consulta profiles joined con roles y dependencies

5.2 Funciones

Función	Descripción
signIn(email, password)	Inicio de sesión
signUp(email, pass, data)	Registro con metadata
signOut()	Cerrar sesión
resetPassword(email)	Enviar recuperación
hasRole(roles)	Verificar rol
isAdmin()	Verificar SUPERADMIN

Función	Descripción
isCoordination()	Verificar COORDINACION
isProfessional()	Verificar PROFESIONAL
isAprendiz()	Verificar APRENDIZ

5.3 Protección de Rutas

- Sin usuario !' /login
- Sin permiso !' /unauthorized
- /dashboard !' APRENDIZ
- /professional !' PROFESIONAL
- /coordination !' COORDINACION, SUPERADMIN
- /admin !' SUPERADMIN

6. Repositorios de Datos

6.1 AppointmentRepository

Método	Retorna	Descripción
fetchAll(filters)	Array<Appointment>	Citas con enriquecimiento
create(data)	Appointment	Crear cita con validación
update(id, updates)	Appointment	Actualizar cita
cancel(id)	Appointment	Cancelar (solo pending)
checkAvailability(dep, date, time)	boolean	Verificar disponibilidad
countPending(userId)	number	Contar pendientes

6.2 ProfessionalRepository



Método	Retorna	Descripción
fetchSchedules(id)	Array<Schedule>	Horarios semanales
createSchedule(data)	Schedule	Crear horario
updateSchedule(id, data)	Schedule	Actualizar horario
deleteSchedule(id)	void	Eliminar horario
fetchDayAgenda(id, date)	Array<Appointment>	Agenda del día
fetchStats(id, month, year)	Object	Estadísticas mensuales
fetchClinicalNotes(apptId)	Array<Note>	Notas de una cita
createClinicalNote(data)	ClinicalNote	Crear nota

6.3 DashboardRepository

Método	Retorna	Descripción
fetchKPIs(filters)	Object	Indicadores clave
fetchDependencyChart(filters)	Array	Citas por dependencia
fetchMonthlyTrend(filters)	Array	Tendencia mensual
fetchProfessionalPerformance(filters)	Array	Ranking profesionales
exportCSV(filters)	string	Generar CSV

6.4 AdminRepository

Método	Retorna	Descripción
fetchUsers(filters)	Array<User>	Lista de usuarios
updateUser(id, data)	User	Actualizar usuario
fetchDependencies()	Array<Dependency>	Lista dependencias

Método	Retorna	Descripción
createDependency(data)	Dependency	Crear dependencia
updateDependency(id, data)	Dependency	Actualizar dependencia
deleteDependency(id)	void	Eliminar dependencia
fetchRoles()	Array<Role>	Roles con permisos
updateRolePermissions(id, perm)	Role	Actualizar permisos
fetchAuditLogs(filters)	Array<AuditLog>	Registros auditoría
fetchSystemConfig()	Object	Configuración
updateSystemConfig(config)	Object	Actualizar config

7. Reglas de Negocio

7.1 Validaciones de Citas

- Máximo 2 citas pendientes por aprendiz
- No fines de semana
- Mínimo 24 horas de anticipación
- Horario: 8:00 AM - 5:00 PM
- Sin doble agendamiento
- Asignación automática de profesional

7.2 Estados de la Cita

Estado	Transiciones	Quién Cambia
pending	!' confirmed, !' cancelled	Profesional, Aprendiz
confirmed	!' completed, !' no_show	Profesional

Estado	Transiciones	Quién Cambia
completed	Ninguna (final)	-
cancelled	Ninguna (final)	-
no_show	Ninguna (final)	-

7.3 Auditoría

- Cada CRUD genera registro con old_data y new_data
- Registros inmutables
- Captura automática de user_agent

8. Componentes Principales

8.1 Compartidos

Componente	Función
DashboardLayout	Shell principal con sidebar y header
Modal	Modal accesible con focus trap
CommandPalette	Ctrl+K para navegación rápida
ProfileDropdown	Panel de perfil y configuración
ErrorBoundary	Captura errores React
ConfirmModal	Diálogo de confirmación
EmptyState	Estados vacíos y skeleton

8.2 Citas

Componente	Función
------------	---------

Componente	Función
AppointmentForm	Formulario Zod-validated
AppointmentCard	Tarjeta con badge de estado
DayAgenda	Línea de tiempo diaria
ClinicalNotes	CRUD de notas clínicas
ScheduleManager	Editor horarios semanales
ProfessionalStats	Estadísticas mensuales

8.3 Dashboard

Componente	Función
KPICard	Tarjeta de métrica
DependencyChart	Gráfico de barras
MonthlyTrendChart	Gráfico de línea
ProfessionalTable	Tabla de rendimiento

8.4 Administración

Componente	Función
UserManagement	CRUD de usuarios
DependenciesManager	CRUD dependencias
RolesPermissions	Editor de permisos
AuditLogViewer	Visor de auditoría
SystemConfig	Configuración del sistema

9. Estilos y Temas

- CSS vanilla con variables CSS en variables.css
- Dark mode con clase .dark y media query
- Persistencia en localStorage
- ThemeProvider gestiona el estado
- Estilos por módulo: auth, appointments, dashboard, admin

10. Testing

10.1 Configuración

- Framework: Vitest
- Testing Library: @testing-library/react
- Entorno: jsdom
- Setup: src/test/setup.js
- Mocks: src/test/mocks/supabase.js

10.2 Convenciones

- Tests con 'should' al inicio
- Estructura Arrange-Act-Assert
- Un test por comportamiento
- Mockear servicios externos
- renderHook para hooks
- Verificar mocks con toHaveBeenCalledWith

11. Instalación

11.1 Prerrequisitos

- Node.js 18+
- npm o yarn
- Cuenta Supabase

11.2 Variables de Entorno

Crear .env.local:

- VITE_SUPABASE_URL=url-del-proyecto
- VITE_SUPABASE_ANON_KEY=clave-anon

11.3 Comandos

Comando	Descripción
npm install	Instalar dependencias
npm run dev	Servidor de desarrollo
npm run build	Build de producción
npm run test	Ejecutar tests
npm run lint	Análisis ESLint

12. Despliegue

- Ejecutar npm run build
- Carpeta dist/ lista para desplegar
- Plataformas: Vercel, Netlify, Cloudflare Pages
- Configurar redirecciones para SPA

13. Convenciones de Código

Elemento	Convención	Ejemplo
Archivos JS	camelCase	appointments.repository.js
Componentes	PascalCase	AppointmentCard.jsx
Hooks	use + Nombre	useAppointments.js
Repositories	Sufijo Repository	AppointmentRepository.js
Tests	Mismo + .test	appointments.repository.test.js

14. Git

14.1 Commits

Formato: tipo(corto): descripción

- feat: Nueva funcionalidad
- fix: Corrección de bug
- docs: Documentación
- refactor: Refactorización
- test: Tests
- chore: Mantenimiento

14.2 Ramas

Rama	Propósito
main	Producción
develop	Desarrollo

Rama	Propósito
feature/*	Nuevas features
fix/*	Correcciones

15. Seguridad

- Supabase Auth con JWT
- .env.local para variables sensibles
- RLS en todas las tablas
- Notas clínicas privadas por defecto
- Auditoría inmutable
- Validación de permisos en cliente y servidor

16. Accesibilidad

- Componentes semánticos HTML5
- aria-labels en botones
- Navegación completa por teclado
- Focus trap en modales
- Contraste WCAG AA
- Skeleton components para carga

17. Performance

- Lazy loading en routes
- useMemo y useCallback para evitar re-renders
- Paginación para listados grandes

- Debounce de 300ms en búsquedas
- Optimistic UI updates
- Realtime para datos frecuentes